

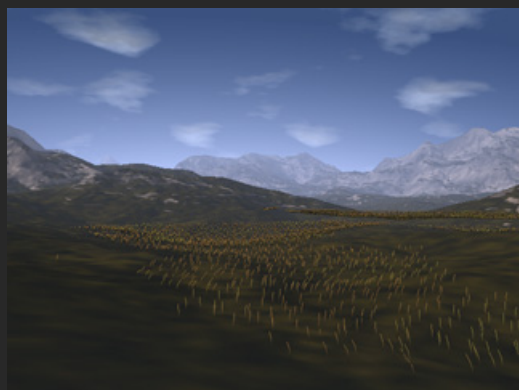
Inigo Quilez :: articles :: 2D dynamic clouds - 2005



Since *Paradise Is Coming* 64kb intro was released back in the 2002 (watch it [here](#)) and I have been asked several times about the algorithm to create the dynamic clouds on that intro.

The intro showed several landscapes with clouds moving and changing shape in the sky. But the interesting point on dynamic clouds is not the fact that you can animate them, since in real life you hardly can see the clouds changing during say, one minute, at least in normal conditions. However, being able to make dynamic clouds basically means you have a procedural way to create them. And this is not only good for 64 kb demos where you obviously need procedural content, but also for those kind of applications where the weather condition shown is not predefined, as interactive virtual reality simulators or scenario editors.

Actually the algorithm I used for that intro is a very well known and documented one. I'm going to write about it anyway, because I will set the basis to explain how it can be modified to completely fit in the graphics card's GPU, what makes the rendering extremely fast (almost for free). In *Paradise Is Coming* the algorithm was completely implemented in the CPU, and I had to trade off some quality to achieve high performance. With the algorithm completely mapped into the GPU, not only faster rendering is possible, but also high quality rendering.

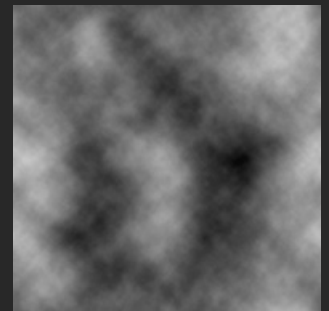
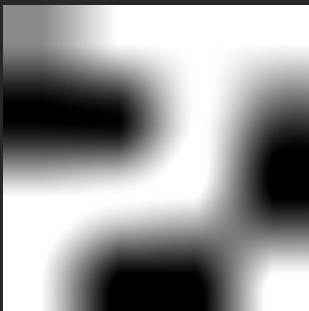


We first assume we are going to create 2D clouds, that will hopefully successfully fake real volumetric clouds. For that we will create one or more 2D layers that will be used as input to the sky dome shader. That shader will also take into account other factors as background stars, sun position, and so on. So, we will concentrate on creating one of those dynamic 2D cloud layers. What we require from the algorithm is basically the ability of the user to define the density of the clouds, and the sharpness at least.

We will use, as expected, some Perlin noise octaves to construct an fbm or turbulence function as basic primitive. The most obvious way to generate dynamic 2D clouds is creating a 3D static cloud volume and use the time to index into the third coordinate. We discard the option of generating the 3D turbulence in real-time since we want the technique to be as cheap as possible (we are interested in the real-time usage of it). Thus, we can think on precomputing it into a 3D texture and then select the correct slice according to time. Doing the linear interpolation between the two closest slices is for free if we use hardware 3D textures to do this (trilinear interpolation). Problem with this option is that having a nice 2D cloud texture (say, 512x512 at least) could easily consume half the video memory if not all of it (in 2005 graphics card's with 512 Megabytes are still rare).

The alternative is a nice trick. Decomposing the turbulence (or fbm) into all the octaves (9 for 512x512 resolution) and store each of them into a texture. This is all we will need for our algorithm, so we reduce the storage by a factor of 50, and even more if we notice that low frequency octaves can probably be stored in smaller resolution. Now, the trick to get a dynamic fbm is to move each octave into the 2D plane with different velocities, and re-compose the turbulence each frame from the individual offseted Perlin noises.

So, first step is to create the N octaves of noise and store them into textures. There are many ways to do this. The easiest is to directly call a procedural Perlin noise routine with the appropriate scale factor (frequency -or scale- doubles each octave). The way I made it for *Paradise Is Coming* was different. N pure noise textures were created, and then a bilinear interpolation based smoothing was applied, with an interpolation size two times smaller each octave. Below are the examples of three of these textures: interpolation of 64, 16 and 4, as well as the final composition:



Here is the pseudo-code to do this as it was implemented in *Paradise Is Coming*, even if just directly filling the

textures with the correct scaled perlin noise is a lot simpler, smarter and smaller (in code):

```
// for every octave: fill the texture
// with noise and interpolate
for( j=0; j<8; j++ )
{
    for( i=0; i<(256*256); i++ )
        text[j][i] = rand()&255;
    interpolate( text[j], 1<<j );
}

void interpolate( unsigned char *buffer, int step )
{
    for( int y=0; y<256; y+= step )
    for( int x=0; x<256; x+= step )
    {
        float a = buffer[(y<<8)+x];
        float b = buffer[(y<<8)+((x+step)&255)];
        float c = buffer[((y+step)&255)<<8)+x];
        float d = buffer[((y+step)&255)<<8)+((x+step)&255)];
        for( int j=0; j<step; j++ )
        for( int i=0; i<step; i++ )
        {
            int xf = (float)i/(float)step;
            int yf = (float)j/(float)step;
            xf = xf*xf*(3.0f-2.0f*xf);
            yf = yf*yf*(3.0f-2.0f*yf);
            buffer[((y+j)<<8)+(x+i)] = a + xf*(b-a) +
                yf*(c-a) + xf*yf*(a-b-c+d);
        }
    }
}
```

Then, at runtime, we have to offset the textures according to each layer's velocity, and add all them together with the appropriate weights (geometrically decreasing with the frequency, to make the fractal feature of the clouds). Offsetting is not really implemented as such, because the same effect can be achieved by just doing dependant texture fetchings. The important thing here is to do the offsetting in a way it doesn't break the illusion of dynamic clouds. For that, it's a good idea to move faster those layers with higher frequency content, ie, the noisiest ones. The direction the layers move is not that critical, though. For *Paradise Is Coming*, layers 0, 3 and 6 were displaced vertically (that's the reason for the 8 bit displacement in the code below) while layers 1, 2, 3, 5 and 7 were displaced horizontally. The final grey level dynamic fbm texture was obtained by just summing up the contributions with the corresponding weights, as shown in the formula below, implemented in fixed point:

```
// offsets for moving the layers
```

```
unsigned int    d[8];

d[0] = (unsigned int)( t*2.5f ); d[0] <= 8;
d[1] = (unsigned int)( t*3.0f );
d[2] = (unsigned int)( -t*3.5f );
d[3] = (unsigned int)( t*4.0f ); d[3] <= 8;
d[4] = (unsigned int)( t*4.5f );
d[5] = (unsigned int)( -t*5.0f );
d[6] = (unsigned int)( t*5.5f ); d[6] <= 8;
d[7] = (unsigned int)( -t*6.0f );
```

```
// composition (synthesis)
```

```
for( i=0; i<65536; i++ )
{
    c = 0;
    for( k=0; k<8; k++ )
    {
        o = i + d[k];
        o &= 0x0000ffff;
        c += (text[k][o] << k);
    }
    c+=128;
    *buffer++ = c>>8;
}
```

$$fbm(x, y, t) = \sum_{i=1}^8 0.5^i \cdot n_i(x + vx_i, y + vy_i)$$

The algorithm on the right executes very quickly even in a old computer, since there is no temporary buffers, the operations are done using integer numbers and not floating point, and because some properties of power-of-two numbers are used. However, because the offsetting of the textures were integer, the animation suffered a bit from artifacts. This issue can be solved by using fixed point offsets and bilinear interpolation when fetching from the noise texture. However, since all the algorithm can be easily moved to the GPU as we will see later, I just let this version work like this and forgot about the issue, since the graphics card is just excellent on doing bilinear interpolations.

Once we have the dynamic 2D fbm we have to use it for creating our cloud layer. We realize we need big cloud-free areas (see first image in this page). However, we might be interested in cloudy skies too (see the image to the right). So, we need an easy way to control the emptiness of clouds. By taking a threshold to our fbm texture to interpret every value below that threshold as empty sky and everything above as cloud, we get what we need. So, we clip every value smaller than the threshold to zero, and for sure, we remap the high part of the fbm (everything above the threshold) to the range 0..1 (0..255). This can be quickly made with a linear mapping. In the case of *Paradise Is Coming*, because there was no interactivity with any user, I could optimize the linear transformations with bit shifts instead of expensive multiplications. As we will see, in the GPU version of the algorithm, however, doing the remapping is as easy as a MAD pixel shader instruction.

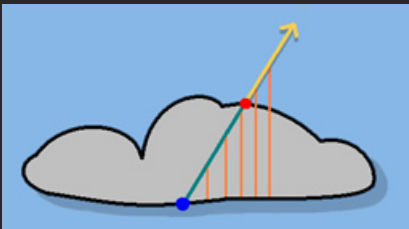
The final step is to use the resulting texture as layer in the sky rendering, for example, to lerp the sky and the cloud color (white and blue in the most basic case).

However, if clouds are implemented just like this, they look quite flat and boring. We need a kind of lighting effect to get some volumetric feeling from them. For that I made a kind of fast raycasting on the cloud, I think in a similar way Terragen is doing. The idea is to consider the cloud to be more dense where the fbm value is higher. If we assume a more or less flat base for the cloud, denser means higher for us. We also assume that we are always looking to the clouds from below, what makes sense because we are simulating them with a 2D layer on the dome. That means that for every pixel in the screen that belongs to a cloud, the light ray going from the camera to the pixel and intersecting the cloud is hitting it in the bottom part of the cloud (the blue dot in the picture below). The light leaving that point towards our eye is the same entering the cloud from the sun direction in red point that crosses the cloud and gets attenuated. We assume that the more cloud volume a ray traverses, the more light attenuation happens. So, light coming from the sun from the red point in the picture will lose energie when traversing the cloud in his way to the blue point, the point we actually render. We are assuming very basic lighting model, with single scattering, but we are not interested in physical correctness, but in having something working.

So, by coloring the pixel with darker color when the sun rays traverse more volume and brighter when it traverses less volume, we get a nice shading that gives volume to the cloud, since thicker parts are shown in darker colors. To quickly determine the distance a ray traverses inside the cloud (green segment) we need a kind of raymarching algorithm. I used a few equally spaced positions along the sun direction ray to sample the cloud altitude (density of the fbm). For points in the ray inside the cloud, the actual vertical position is smaller than the cloud altitude, while the contrary holds for points outside the volume. So, doing a comparison per point allows to estimate the distance the ray traverses the cloud.

For example, if the actual pixel was in the right part of the cloud, just one

of the 5 samples happened to be inside probably. The proportion of samples inside the cloud was used as gray level for the shading. Tweaking the contrast and softening of this value, the shading was made.



Moving the clouds to the graphics card

The clouds technique described above was developed in 2002, before GPU shaders existed. Soon after that, it became usual in real-time graphics community to move to the GPU algorithms that where traditionally executed on the CPU. And so I did also with the 2D dynamic clouds. Since the basic algorithm is offsetting some precomputed noise textures and adding them together, the algorithm is very suitable for the GPU.

As in the CPU version, first the N layers of noise must be backed into textures. However, for the GPU version I used four layers instead of eight, even if my final cloud layer was 512x512. Textures could normally be grey level textures; however, to speed up the raymarching process I described in the CPU version, I used the four RGBA channels to do four texture fetches into a single one. I will explain this later, lets go first to the fbm

construction.

To calculate the cloud texture we need to render into a screen aligned quad of 512x512 pixels. We bind the four noise textures to it, and we give the usual texture coordinates (0,0), (0,1), (1,0) and (1,1). In the vertex shader we will offset this texture coordinates to simulate the noise offsetting. This works as long as we are using a WRAP texturing mode, and not CLAMP. The texture coordinates interpolation and bilinear interpolation circuitry take the responsibility of creating a smooth layer movement. Thus, the vertex shader makes nothing else than writing the output clip space vertex position and calculating the texture coordinates. For that last task, the basic (0,0)-(0,1)-(1,0)-(1,1) texture coordinate is passed in the channel 0, and the rest are calculated based on the per-layer speed that are given as local variables to the shader. Doing this computation in the vertex shaders saves to the pixel shader from doing dependent texture reads, what is good for performance.

```
// vertex shader
!!ARBvp1.0"
OPTION ARB_position_invariant;
ADD result.texcoord[0], program.local[0], vertex.texcoord[0];
ADD result.texcoord[1], program.local[1], vertex.texcoord[0];
ADD result.texcoord[2], program.local[2], vertex.texcoord[0];
ADD result.texcoord[3], program.local[3], vertex.texcoord[0];
END;
```

The fragment shader samples the four noise textures based on these texture coordinates and adds them with the corresponding weights to get the fbm value:

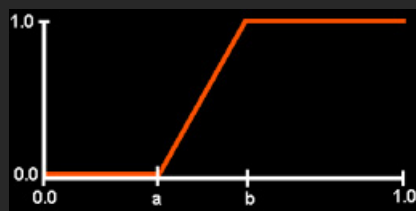
```
// fragment shader
TEX n1, fragment.texcoord[0], texture[0], 2D;
TEX n2, fragment.texcoord[1], texture[1], 2D;
TEX n3, fragment.texcoord[2], texture[2], 2D;
TEX n4, fragment.texcoord[3], texture[3], 2D;

MUL fbm, 0.5000, n1;
MAD fbm, 0.2500, n2, fbm;
MAD fbm, 0.1250, n3, fbm;
MAD fbm, 0.0625, n4, fbm;
```

Next, we have to do the fbm remapping to get the empty sky regions. As explained for the CPU implementation, a linear mapping makes the job quite well. The MAD instruction perfectly meets our needs, since it implements the line equation $y(x) = m \cdot x + m$. By adding the _SAT suffix to the instruction we convert the line into something like the graph shown below. So, we have two values we can tweak, m and n . However it's more intuitive to tune the values a and b on the graph, since the first one is the threshold for cloud/empty-sky proportion (the lower, the more cloud we will have), and the second one helps to control the sharpness of the clouds (the closer b gets to a , the sharper the clouds will be). The conversion from a/b to m/n is quite

simple: $m = 1/(b-a)$ and $n = -a/(b-a)$. We can pass this values to the shader in program local variables and allow dynamic changes of weather conditions (going from cloudy skies to clear ones):

```
MAD_SAT fbm, fbm, program.local[0].x, program.local[0].y;
MOV result.color.w, fbm.w;
```



If the regular noise values were stored into the *w* component of the texture colors (alpha channel), the *fbm* value would result to be also in the *w* component. Thus we can already output it to the color of the shader so that the synthesized texture has the cloud/sky value into the alpha channel and the sky-dome shader can use it to correctly lerp between sky and cloud color. This allows to have the sky coloring algorithm into another shader, and be as complex as desired but still independent from the cloud generation method.

So, we now have to calculate the actual cloud color (the grey scale coloring due to volumetric nature of the clouds) and put it into the *rgb* channels, ie, we have to do the raymarching.

Because we want the shader to be as fast as possible, we will do it with a trick. We will use only four samples for the raymarching, and we will sample all of them at once. Remember that what we need is the value of the *fbm* in four points: the actual one (the blue one in the picture we saw), and the following three. The current one we already have it in *fbm.w*. Nice trick could be to have the other three in *fbm.xyz...* By inspecting the algorithm we see that it could be already the case if we had our primitive noise textures correctly created: if the *RGB* channels of the each noise texture contained the noise values of the current texture after offsetting the sampling point in the sun direction vector projected in the dome geometry. Basically, it means we have to put into the alpha channel of our noise textures the real noise value, into the blue channel the noise displaced by a raymarching-step (projected into the dome), into the green one the noise displaced by two steps, and into the red one the noise displaced by three steps. Then, without changing the shader's code above we have already the four cloud altitudes in the *fbm* variable ready to use!

The altitude comparisons now become really simple. Put into another variable *ray* the four altitudes of the ray starting from the blue point into the sun direction. Now a simple subtraction of *ray* and *fbm* will create negative values for points inside the cloud, and positive values for points outside. With a correct *MAX* operation or optionally a *MUL_SAT* we can obtain binary values. The final integration (doing the average of them, ie, adding and then dividing by four) can be quickly done with a dot product operation *DP4* between the values and the constant vector (0.25, 0.25, 0.25, 0.25).

Finally, this grey level value can be used to lerp between two user defined colors (passed as local parameters) to tune the cloud colors, since you probably want to mix two dark grey colors for stormy skies while you prefer pure white and medium grey for clear skies). The complete fragment shader then becomes:

```
// fragment shader
!!ARBfp1.0
TEMP fbm, de, n1, n2, n3, n4;
PARAM ray = {0.0, 0.3, 0.6, 0.9};
// calc fbm
TEX n1, fragment.texcoord[0], texture[0], 2D;
TEX n2, fragment.texcoord[1], texture[1], 2D;
TEX n3, fragment.texcoord[2], texture[2], 2D;
TEX n4, fragment.texcoord[3], texture[3], 2D;

MUL fbm, 0.5000, n1;
MAD fbm, 0.2500, n2, fbm;
MAD fbm, 0.1250, n3, fbm;
MAD fbm, 0.0625, n4, fbm;

// remaping: scale, bias, threshold
MAD_SAT fbm, fbm, program.local[0].x, program.local[0].y;
MOV result.color.w, fbm.w;

// ray-marching
ADD de, fbm, -ray;
MAX de, de, 0;

// integration : de.w = 0.25*(de.x+de.y+de.z+de.w)
DP4 de.w, de, {.25, .25, .25, .25};

// calc cloud color
LRP result.color.xyz, de.w, program.local[2], program.local[1];
END;
```

As shown, the implementation of the 2D dynamic clouds can be done in a very small pixel shader, and thus runs very fast. There is no dependent texture reads, nor complex arithmetic instruction. The result is that the clouds don't render slower than a thousand frames per second in a normal graphics card, with subpixel accuracy :) Below some images of the sky rendered with this technique into a real-time demo featuring complete weather simulation.



inigo quilez - learning computer graphics since 1994